

泛型（类属）程序设计--模板

问题：在程设中，会用到一些功能完全相同的程序实体，但它们所涉及的数据类型不同。

1. 一个程序实体能对**多种类型的数据**进行操作或描述的特性称为**类属或泛型**（Generics）

基于具有类属性的程序实体进行程序设计的技术称为：泛型程序设计（或类属程序设计，Generic Programming）

具有类属性的程序实体通常有：

类属函数：一个能对**不同类型的数据完成相同操作**的函数

类属类：一个**成员类型可变、但操作不变**的类

2. 类属函数：

c++提供了两种实现方法：采用通用指针类型的参数（c语言的做法）、
函数模板

- 用通用指针参数实现类属排序函数：

```
1  typedef unsigned char byte;
2  void sort(void *base, //需排序的数据（数组）内存首地址
3          unsigned int num, //数据元素的个数
4          unsigned int element_size, //一个数据元素所占内存大小（字节数）
5          bool (*cmp)(const void *, const void *) ) //比较两个元素的函数
6  { //不论采用何种排序算法，一般都需要对数组进行以下操作
7      //取第i个元素（通过数组首地址、偏移量i以及元素内存大小算出元素地址）
8      (byte *)base+i*element_size
9      //比较第i个和第j个元素的大小（利用调用者提供的回调函数cmp实现）
10     if (!cmp((byte *)base+i*element_size, (byte
11     *)base+j*element_size))
12     { //交换第i个和第j个元素的位置（把两个元素逐个字节进行交换）
13         byte *p1=(byte *)base+i*element_size,
14             *p2=(byte *)base+j*element_size;
15         for (int k=0; k<element_size; k++)
16             { byte temp=p1[k]; p1[k] = p2[k]; p2[k] =
17             temp;
18         }
19     }
20     //int型数组排序,其他类型的数据排序同下
21     //先定义一个对int型数据进行比较的函数
22     bool int_compare(const void *p1, const void *p2)
23     //比较int类型元素大小。
24     { if (*(int *)p1 < *(int *)p2)
25         return true;
26         else
27         return false;
28     }
29     .....
30     int a[100];
31     .....
32     sort(a,100,sizeof(int),int_compare);
```

缺点：比较麻烦，需要大量的指针操作；容易出错，编译程序无法进行类型检查。

- 函数模板：函数模板是指带有类型参数的函数定义，格式如下：`template <class T1, class T2, ...>`

`<返回值类型> <函数名>(<参数表>)`

```
{ .....  
}
```

T1、T2等是函数模板的参数，它们的取值是某个类型，class也可以写成typename。

函数的<返回值类型>、<参数表>中的参数类型以及函数体中的局部变量的类型可以是：T1、T2等。

- 用函数模板实现类属排序函数

```
1 //排序函数模板的定义：  
2 template <class T>  
3 void sort(T elements[], unsigned int count)  
4 {  
5     //取第i个元素  
6     elements[i]  
7     //比较第i个和第j个元素的大小  
8     if (!(elements[i] < elements[j]))  
9     { //交换第i个和第j个元素  
10        T temp=elements [i];  
11        elements[i] = elements [j];  
12        elements[j] = temp;  
13    }  
14 }  
15  
16 //排序函数模板的使用：  
17 int a[100];  
18 sort(a,100); //对int类型数组进行排序  
19  
20 double b[200];  
21 sort(b,200); //对double类型数组进行排序  
22
```

- 函数模板的实例化：

函数模板定义了一系列**重载的函数**。

要使用函数模板所定义的函数，首先必须要对函数模板进行实例化：

- 给模板参数提供一个具体的类型，从而生成具体的函数。

函数模板的实例化通常是**隐式的**：

- 由编译程序根据函数调用的**实参类型**自动地把函数模板实例化为具体的函数。
- 这种确定函数模板实例的过程叫做**模板实参推导**（template argument deduction）。

```
1 int a[100];  
2 sort (a,100);  
3 //实例化：  
4 void sort(int elements[], unsigned int count) { ..... }  
5  
6 //带比较函数的排序模板
```

```

7  template <class T1, class T2>
8  void sort(T1 elements[], unsigned int count, T2 cmp)
9  { .....
10     //比较第i个和第j个元素的大小
11     if (!cmp(elements[i],elements[j]))
12     { //交换元素次序 }
13     .....
14 }
15 int a[100];
16 sort (a,100,[](int &x1,int &x2) { return x1<x2;});
17 double b[200];
18 sort (b,200,[](double &x1,double &x2) { return x1<x2;});
19 A c[300];
20 sort(c,300,[](A &x1,A &x2) { return x1<x2;});
21

```

有时，编译程序无法根据调用时的实参类型来确定所调用的模板实例函数。如：

```

1  template <class T>
2  T max(T a, T b)
3  { return a>b?a:b;
4  }
5  .....
6  int x,y,z;
7  double l,m,n;
8  z = max(x,y); //实例化和调用: int max(int,int)
9  l = max(m,n); //实例化和调用: double max(double,double)
10 //问题: max(x,m)如何实例化?
11 //显式类型转换:
12 max((double)x,m); //调用: double max(double a,double b)
13 //显式实例化:
14 max<double>(x,m); //或max<int>(x,m)
15

```

- 带非类型参数的函数模板

```

1  template <class T, int size> //size为一个int型的普通参数
2  void f(T a)
3  { T temp[size];
4    .....
5  }
6  //这样的函数模板在使用时需要显式实例化。例如，
7  f<int,10>(1); //调用模板函数f(int a)，其中的size为10
8

```

- 类模板：如果一个类定义中用到的类型可变、但操作不变，则该类称为类属类。
类模板的格式为：template <class T1,class T2,...> //class也可以写成typename
class <类名>
{<类成员说明>
}
用类模板实现类属的栈类：

```

1  template <class T>
2  class Stack
3  {      T buffer[100];
4          int top;
5      public:
6          Stack() { top = -1; }
7          void push(const T &x);
8          void pop(T &x);
9  };
10 template <class T>
11 void Stack <T>::push(const T &x) { ..... }
12 template <class T>
13 void Stack <T>::pop(T &x) { ..... }
14

```

- 类模板的实例化：需要在程序中显式指出，如：

```

1  Stack<int> st1; //实例化int型栈类并创建一个相应类的对象
2  int x;
3  st1.push(10); st1.pop(x);
4
5  Stack<double> st2; //实例化double型栈类并创建一个相应类的对象
6  double y;
7  st2.push(1.2); st2.pop(y);
8
9  //注意：一个类模板的不同实例之间不共享该类模板中的静态成员
10

```

- 带非类型参数的类模板

3. 模板的复用：模板也属于一种多态，称为**参数化多态**

- 一段带有类型参数的代码，给该参数提供不同的类型就能得到多个不同的代码，即，**一段代码有多种解释**。
- 模板的复用是通过对**模板进行实例化（用一个具体的类型去替代模板的类型参数）**来实现的：
- 由于**实例化是在编译时刻进行的**，它一定要见到**相应的源代码**，因此，模板属于**源代码复用**。
- 一个模板有很多的实例，是否实例化模板的某个实例，这要由使用情况来决定。

在编程时最好把模板的定义和实现都放在头文件中。使用者通过包含这个头文件，把模板的源代码全包含进来，以备实例化所需。

模板是基于源代码的复用！

- 在由多模块构成的程序中，由于每个模块是单独编译的，因此，会导致一个模板的某个实例存在于多个模块的编译结果中：相同的函数模板实例，相同的类模板成员函数实例。
- 如何处理相同的实例？
 - 由开发环境来解决：编译第二个模块的时候不生成这个实例。（代价大！）
 - 由链接程序来解决：舍弃一个相同的实例。

4. 类模板的友元函数：

```

1  template <class T> //类模板A的定义
2  class A
3  { T x,y;
4      .....

```

```

5     friend void f(A<T>& a); //f是多个重载的函数，
6         //它们与A的实例是一对一友元！
7 };
8 void f(A<int>& a) { ..... } //该f仅是A<int>的友元，
9 void f(A<double>& a) { ..... } //该f仅是A<double>的友元
10 .....
11 A<int> a1; //实例化A<int>
12 A<double> a2; //实例化A<double>
13 A<char *> a3; //实例化A<char *>
14 f(a1); //调用f(A<int>&)
15 f(a2); //调用f(A<double>&)
16 f(a3); //调用f(A<char *>&), 但连接时出错，该函数不存在！
17
18 template <class T> class A; //类模板A的声明（f的定义中要用到）
19 template <class T> void f(A<T>& a) { ..... } //f是函数模板
20
21 template <class T> //类模板A的定义
22 class A
23 { T x,y;
24     .....
25     template <class T1> friend void f(A<T1>& a); //整个模板f是友元，
26         //f的实例与A的实例是多对多友元
27 };
28 .....
29 A<int> a1; //实例化A<int>
30 A<double> a2; //实例化A<double>
31 A<char *> a3; //实例化A<char *>
32 f(a1); //实例化f<int>并调用之，它是A所有实例的友元
33 f(a2); //实例化f<double>并调用之，它是A所有实例的友元
34 f(a3); //实例化f<char *>并调用之，它是A所有实例的友元
35

```